User Story Generator

System Architecture Documentation: User Story Generator

This document provides a comprehensive technical overview of the **User Story Generator** system architecture. The platform is designed to securely ingest multi-modal enterprise data, preprocess and validate the content, and utilize an advanced hybrid retrieval and multi-agent orchestration layer to generate high-fidelity Business Analysis (BA) artifacts (Epics, User Stories, and Traceability Matrices).

1. Input & Connector Layer

The perimeter of the architecture manages multi-source ingestion, establishing secure handshakes with enterprise data repositories.

- **Supported Input Formats & Platforms:**
  - *Unstructured Files:* PDF, Word, Excel, Voice recordings.
  - *Enterprise Systems:* Jira, Azure DevOps, Confluence, SharePoint, Google Drive, and custom integrations via REST APIs.
- **Connector Layer:** A centralized ingestion engine responsible for authentication (OAuth, API keys), rate limiting, and secure data extraction. It handles scheduled syncs and webhook-based real-time triggers to pull data into the processing pipeline.

2. Parsing & Preprocessing Layer

Data flows from the connectors into a deterministic, vertical preprocessing pipeline to transform raw data into uniform, enriched text fragments.

- **Document Parsing (Docling):** Extracts raw text, structural tables, embedded metadata, and establishes the visual/structural document hierarchy.
- **Speech-to-Text (Whisper X):** Transcribes voice inputs (e.g., stakeholder interview recordings) into highly accurate timestamped text.
- **Normalization:** Strips out formatting noise, standardizes character encodings (UTF-8), and cleanses extracted text to optimize downstream NLP performance.
- **Structural Chunking:** Breaks text down into content-aware chunks (target size: **250–300 tokens**). This stage maps parent-child chunk relationships, manages token overlap to preserve context, runs duplicate detection, and executes initial inconsistency checks.
- **Chunk JSON Object Creation:** Structures each chunk into a standardized schema containing: `Chunk ID`, `Document ID`, `Project ID`, `Section Title`, `Content`, `Token Count`, `Context`, `Content Hash`, `Metadata`, and `Created At`.

- **Context Labeling:** Enriches the JSON object with functional identifiers such as *Feature Label*, *Module Label*, *Priority*, *Document Section*, and a *Traceability ID*.
- **NLP Metadata Enrichment:** Uses lightweight NLP models to extract entity details, classify requirements, isolate actors, define keywords, and flag initial risks.
- **Ingestion Coverage Validation:** A critical guardrail that maps processed chunks back to the raw source data. It calculates a *Coverage Percentage* and flags any missing requirements before data storage occurs.

3. Storage & Retrieval Layer

Once validated, data is persisted across distinct storage systems tailored for relational integrity, vector searches, and high-speed caching.

- **PostgreSQL (System of Record):** The relational backbone of the platform. It securely houses the primary **Requirement Inventory**, **Chunk JSON structures**, the **Traceability Matrix**, application metadata, and a comprehensive **Review History** for audit trails.
- **BGE Embedding Generation:** Passages from the validated chunks are passed through a BGE embedding model to generate dense vector representations.
- **Qdrant Vector Database (Dense Vector Storage):** Stores the generated BGE embeddings to enable semantic and contextual mathematical similarity searches.
- **Redis Cache (Secondary Side Database):** Acts as a high-speed volatile cache storing frequently accessed text chunks to ensure low-latency retrieval, fast response times, and an efficient fallback mechanism.

4. Hybrid Retrieval & Core Orchestration

This centralized engine represents the core intelligence of the platform, running within a **LangGraph Orchestration + Groq API** bounding box. It handles data fetching and coordinates sequential AI agents.

A. The Retrieval Pipeline

Before agents execute, a multi-stage retrieval strategy aggregates relevant contexts:

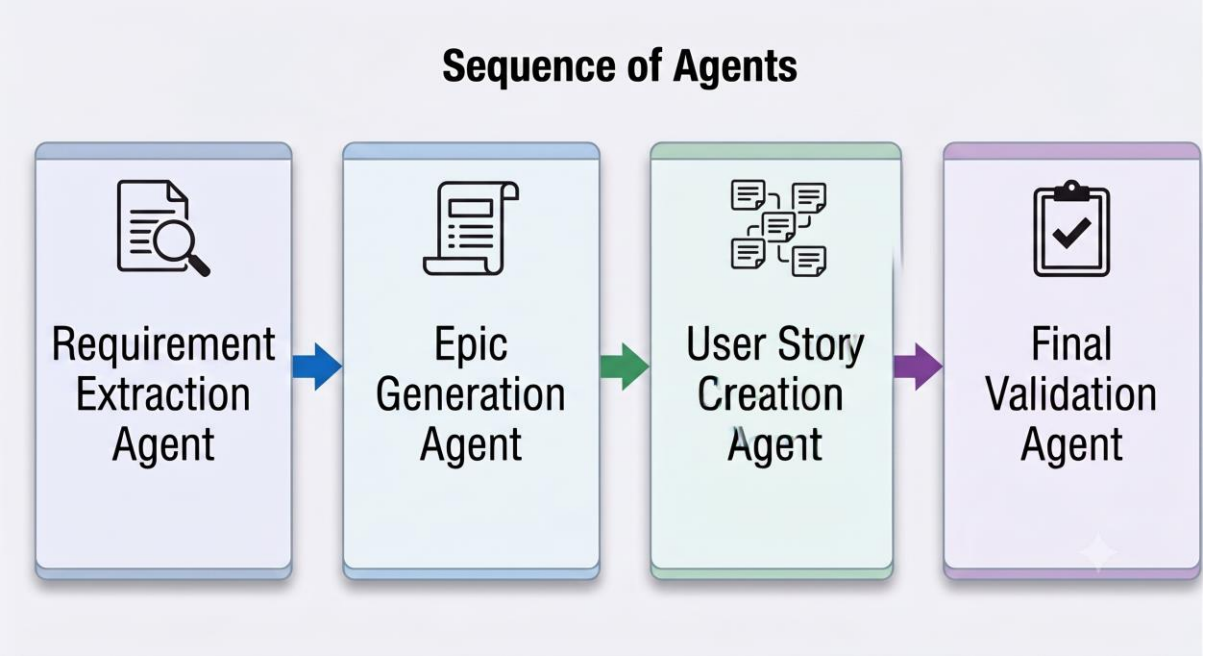
1. **BM25 Retrieval:** Executes exact-match keyword queries against the PostgreSQL Full Text Search database utilizing `tsvector`, `tsquery`, and `GIN` indexing.
2. **Dense Vector Retrieval:** Executes contextual and semantic searches against the Qdrant database.
3. **Reciprocal Rank Fusion (RRF):** Blends the disparate scoring systems of BM25 and Dense Retrieval into a single, unified ranked list.

4. **BGE Cross-Encoder Reranker:** Evaluates the combined RRF results using a deep cross-encoder model to determine actual relevance, outputting the absolute **Top Relevant Context** to feed the agent matrix.

B. LangGraph Agent Matrix

Four sequential agents collaborate to turn refined context into final business analysis documentation:

```
[Top Relevant Context]
          |
          ▼
```



- **Requirement Extraction Agent :** Uses LLM-guided prompts to isolate actors, functional requirements, non-functional requirements, business rules, constraints, and edge cases. Includes automated prompt validation checkpoints.
- **Epic Generation Agent :** Synthesizes extracted requirements into high-level Epics (`Epic ID`, `Title`, `One-line Story Summary`). Maps structural traceability and features a **BA Manual Review & Edit Loop** that dynamically updates the PostgreSQL Traceability Matrix based on human input.
- **User Story Creation Agent:** Generates granular User Stories (`US001`, `US002`...) complete with structured **Acceptance Criteria** and an algorithmic **Confidence Score**. It handles dependency detection, provides up to 3 automated retries for

editing/deletion, and throws a *Dependency Suggestion Warning* popup if logical conflicts occur.

- **Final Validation Agent:** Compares all generated user stories against the original *Requirement Inventory* and raw retrieved chunks. It verifies the final traceability, computes an **Overall Confidence Score**, validates coverage, and halts at a final **BA Manual Review & Approval Checkpoint** before publishing.

5. Export & Integration Layer

Approved artifacts are transformed and synchronized with downstream enterprise environments.

- **File Exports:** Clean generation of Word Documents, Excel Spreadsheets, and formatted PDF Reports.
- **Enterprise Integration (Jira & Azure DevOps):** Provides **bidirectional synchronization**. Changes made in the User Story Generator push directly to your project management tracking software, and updates made inside Jira or Azure DevOps reflect back within the platform's trace matrix.
- **REST API Output:** Exposes structured endpoints for external third-party or custom enterprise applications to securely fetch the generated requirements pipeline.